

ECS - ELASTIC CONTAINER SERVICE

1. Setup a highly available ECS cluster with load balancer and dynamic port mapping.

NOTE:

Your cluster should maintain atleast 6 tasks at any point of time and should be highly available across multiple AZ's.

Use the below image from deployment: sabair0509/hiring-app:works

1. Architecture we will create

Use these components:

Component	Configuration
ECS Cluster	hiring-ecs-cluster
Launch type	EC2
EC2 instances	3 minimum
Availability Zones	3 AZs if available
Auto Scaling Group	Min 3 / Desired 3 / Max 6
Capacity Provider	hiring-capacity-provider
ECS Service	hiring-service
Desired tasks	6
Minimum tasks	6
ALB	hiring-alb
Target Group	hiring-tg
Listener	HTTP 80
Container port	We'll verify from image
Host port	0 — dynamic

Component Configuration

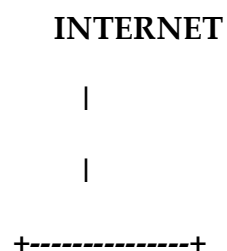
Docker image sabair0509/hiring-app:works

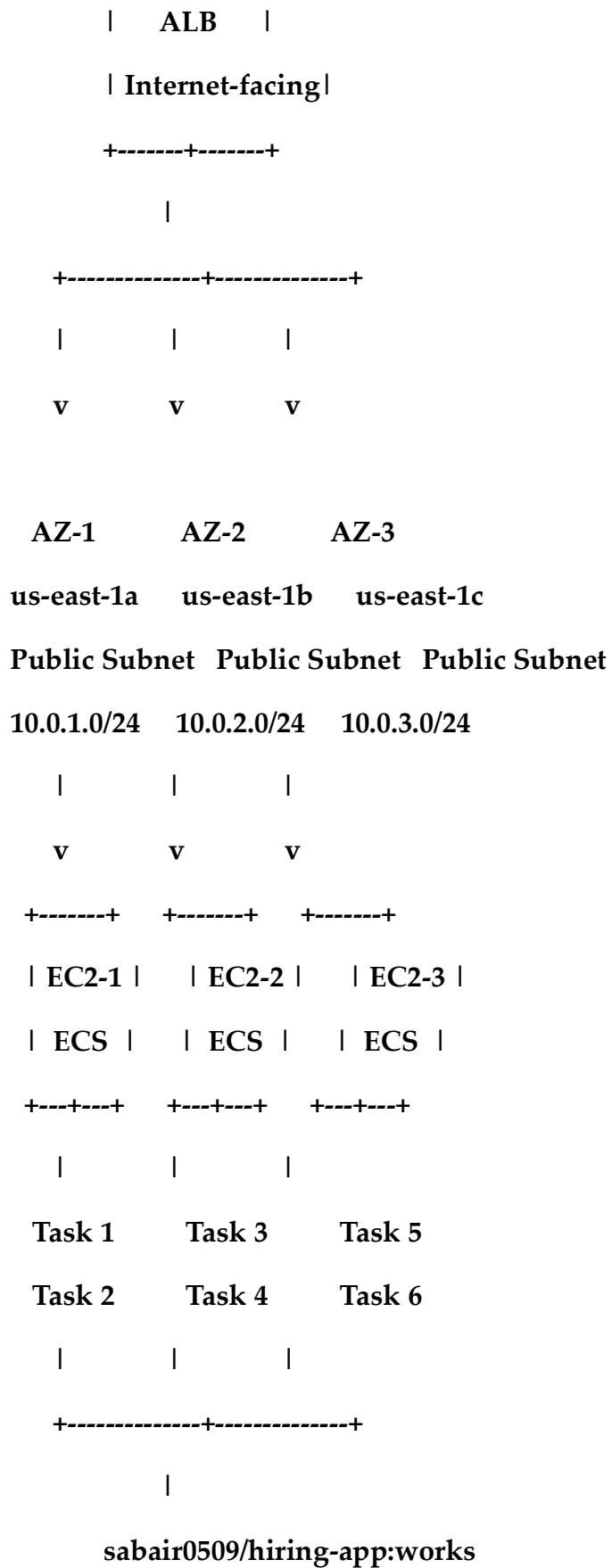
With EC2 ECS, ALB supports dynamic host-port mapping: container port can be fixed while host port is set to 0, causing ECS to select an available ephemeral port.

Your setup should be

Component	How many?	Purpose
VPC	1	Entire ECS network
Availability Zones	3	High availability
Public Subnets	3	One subnet per AZ
EC2 instances	3	One ECS container instance per AZ
ECS Cluster	1	Runs your ECS tasks
ALB	1	Distributes traffic
Target Group	1	Tracks ECS task IP/ports
ECS Service	1	Maintains 6 tasks
ECS Tasks	6 minimum	Your requirement
Auto Scaling Group	1	Maintains EC2 capacity
Capacity Provider	1	Connects ECS to ASG

Network layout





With EC2 ECS, ALB supports dynamic host-port mapping: container port can be fixed while host port is set to 0, causing ECS to select an available ephemeral port.

2. Choose your AWS Region

I recommend using:

us-east-1

Make sure you have at least **3 Availability Zones** available.

For example:

us-east-1a

us-east-1b

us-east-1c

You can check:

AWS Console → VPC → Subnets

3. Create / verify VPC

You can use an existing VPC if you already have one.

For this assignment, ideally have:

VPC

|

+-- Public Subnet AZ-1

|

+-- Public Subnet AZ-2

|

+-- Public Subnet AZ-3

For a lab assignment, you can place ECS EC2 instances in public subnets.

Example:

VPC: 10.0.0.0/16

AZ-1

Public subnet: 10.0.1.0/24

AZ-2

Public subnet: 10.0.2.0/24

AZ-3

Public subnet: 10.0.3.0/24

Ensure:

- Internet Gateway attached
- Route table has 0.0.0.0/0 → Internet Gateway
- Auto-assign public IPv4 enabled

STEP 1: Select AWS Region

Use:

us-east-1

You need at least **3 Availability Zones**:

us-east-1a

us-east-1b

us-east-1c

STEP 2: Create One VPC

Go to:

AWS Console

→ VPC

→ Your VPCs

→ Create VPC

Select:

Resources to create: VPC only

Enter:

Name: hiring-ecs-vpc

IPv4 CIDR: 10.0.0.0/16

Click:

Create VPC

The screenshot shows the 'Create VPC' page in the AWS Management Console. The left sidebar contains the 'VPC dashboard' with a search bar and a list of VPC-related services. The main content area is titled 'Create VPC' and includes a description of VPCs. Under 'VPC settings', the 'Resources to create' section has 'VPC only' selected. The 'Name tag - optional' field contains 'hiring-ecs-vpc'. The 'IPv4 CIDR block' section has 'IPv4 CIDR manual input' selected, with the CIDR '10.0.0.0/16' entered. The 'IPv6 CIDR block' section has 'No IPv6 CIDR block' selected.

The screenshot shows the 'VPC details' page for the VPC 'vpc-084b1b829186eae2d / hiring-ecs-vpc'. A green banner at the top indicates 'You successfully created vpc-084b1b829186eae2d / hiring-ecs-vpc'. The 'Details' section displays various attributes: VPC ID (vpc-084b1b829186eae2d), State (Available), Block Public Access (Off), DNS hostnames (Disabled), DNS resolution (Enabled), Tenancy (default), Default VPC (No), Network Address Usage metrics (Disabled), Encryption control ID (None), DHCP option set (dopt-0cffe704340cfdde1), IPv4 CIDR (10.0.0.0/16), Route 53 Resolver DNS Firewall rule groups (None), Main network ACL (acl-088083d01ab5f1de0), Main route table (rtb-06a83aae989ecb8da), IPv6 pool (None), and Owner ID (571833381790). Below the details is a 'Resource map' section showing a visual representation of the VPC, Subnets (0), and Route tables (1).

STEP 3: Create 3 Public Subnets

Go to:

VPC

→ Subnets

→ Create subnet

Select:

VPC: hiring-ecs-vpc

Subnet 1

Name: public-subnet-1a

Availability Zone: us-east-1a

CIDR: 10.0.1.0/24

Subnet 2

Name: public-subnet-1b

Availability Zone: us-east-1b

CIDR: 10.0.2.0/24

Subnet 3

Name: public-subnet-1c

Availability Zone: us-east-1c

CIDR: 10.0.3.0/24

The assignment guide uses exactly this type of one-VPC, three-AZ subnet layout.

The screenshot displays the AWS Management Console interface for the 'Subnets' page. The top navigation bar shows the AWS logo, a search bar, and the user's profile. The left sidebar contains the navigation menu, with 'Subnets' selected under the 'Virtual private cloud' section. The main content area shows a list of subnets for the VPC 'vpc-084b1b829186eae2d'. The subnets are 'pub-sub-1a', 'pub-sub-1b', and 'pub-sub-1c', all with a state of 'Available'. The table includes columns for Name, Subnet ID, State, VPC, and Block Public Access. Below the table, there is a 'Select a subnet' dropdown menu.

Name	Subnet ID	State	VPC	Block Public...
pub-sub-1a	subnet-0e6c8ab7b802da5c8	Available	vpc-084b1b829186eae2d hiring-ecs-vpc	Off
pub-sub-1b	subnet-0168d13759fa3afa3	Available	vpc-084b1b829186eae2d hiring-ecs-vpc	Off
pub-sub-1c	subnet-0a23a9d0957222c0f	Available	vpc-084b1b829186eae2d hiring-ecs-vpc	Off

STEP 4: Create Internet Gateway

Go to:

VPC

→ Internet Gateways

→ Create internet gateway

Name:

hiring-ecs-igw

Click:

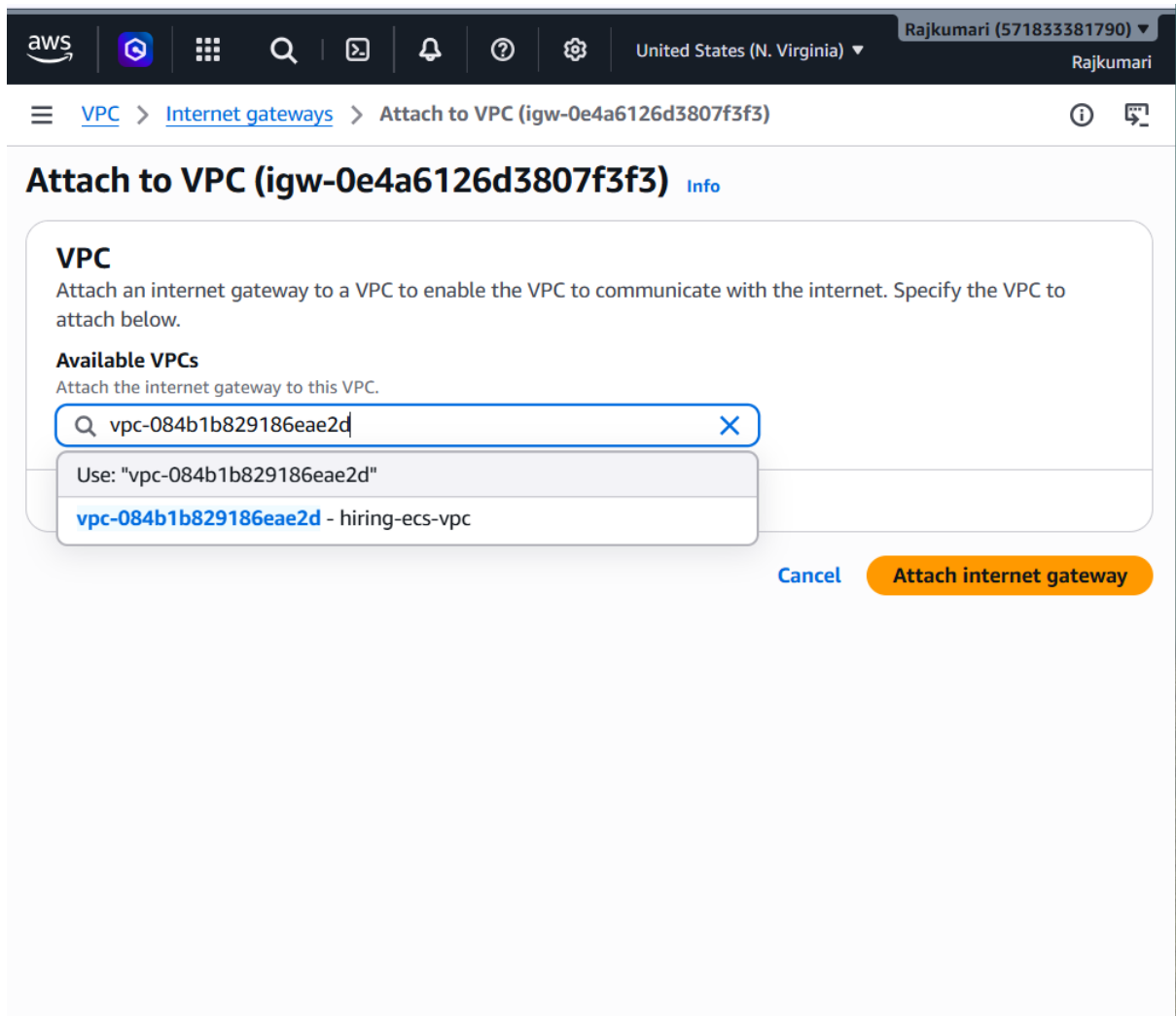
Create

Then:

Actions

→ Attach to VPC

→ hiring-ecs-vpc



STEP 5: Create/Configure Route Table

Go to:

VPC

→ Route Tables

Select the route table associated with your VPC.

Go to:

Routes

→ Edit routes

→ Add route

Add:

Destination: 0.0.0.0/0

Target: Internet Gateway

Target: hiring-ecs-igw

Save.

Then go to:

Subnet associations

→ Edit subnet associations

Select:

public-subnet-1a

public-subnet-1b

public-subnet-1c

Save.

aws         United States (N. Virginia)  Rajkumari (571833381790)  Rajkumari

VPC > Route tables > rtb-06a83aae989ecb8da > Edit subnet associations

Edit subnet associations

Change which subnets are associated with this route table.

Available subnets (3/3)

Filter subnet associations

☒	Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
☒	pub-sub-1a	subnet-0e6c8ab7b802d...	10.0.1.0/24	–
☒	pub-sub-1b	subnet-0168d13759fa3...	10.0.2.0/24	–
☒	pub-sub-1c	subnet-0a23a9d095722...	10.0.3.0/24	–

Selected subnets

subnet-0e6c8ab7b802da5c8 / pub-sub-1a 

subnet-0168d13759fa3afa3 / pub-sub-1b 

subnet-0a23a9d0957222c0f / pub-sub-1c 

Cancel Save associations

STEP 6: Enable Auto-Assign Public IPv4

For each subnet:

VPC

→ Subnets

→ Select subnet

→ Actions

→ Edit subnet settings

Enable:

Auto-assign public IPv4 address

Repeat for all 3 subnets.

Edit subnet settings [Info](#)

Subnet

Subnet ID	Name
subnet-Oe6c8ab7b802da5c8	pub-sub-1a

Auto-assign IP settings [Info](#)

Enable AWS to automatically assign a public IPv4 or IPv6 address to a new primary network interface for an instance in this subnet.

☒ Enable auto-assign public IPv4 address [Info](#)

☐ Enable auto-assign customer-owned IPv4 address [Info](#)
Option disabled because no customer owned pools found.

Resource-based name (RBN) settings [Info](#)

Specify the hostname type for EC2 instances in this subnet and optional RBN DNS query settings.

☐ Enable resource name DNS A record on launch [Info](#)

☐ Enable resource name DNS AAAA record on launch [Info](#)

Hostname type [Info](#)

☐ Resource name

☒ IP name

STEP 7: Create ECS EC2 IAM Role

Go to:

IAM

→ Roles

→ Create role

Select:

Trusted entity type: AWS service

Choose:

Use case: EC2

Attach the ECS container-instance policy available in your account, as indicated by the guide's ECS EC2 instance-role setup. The guide names the role:

ecsInstanceRole

Create the role:

Role name: ecsInstanceRole

The screenshot shows the AWS IAM console interface for creating a new role. The top navigation bar includes the AWS logo, a search bar, and the user's name 'Rajkumari (571833381790)'. The breadcrumb trail indicates the path: IAM > Roles > Create role. On the left, a progress bar shows three steps: 'Step 1: Select trusted entity', 'Step 2: Add permissions', and 'Step 3: Name, review, and create', with the third step being the active one. The main content area is titled 'Name, review, and create'. Under 'Role details', the 'Role name' field contains 'ecsInstanceRole' and the 'Description' field contains 'Allows EC2 instances to call AWS services on your behalf.' Below this, 'Step 1: Select trusted entities' is shown with an 'Edit' button. The 'Trust policy' section displays a JSON snippet:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
```

This screenshot shows the 'Roles' page in the AWS IAM console. A green banner at the top states 'Role ecsInstanceRole created.' with a 'View role' link. The 'Roles (7)' section includes a search bar and a table listing roles. The table has columns for 'Role name', 'Trusted entities', and 'Last activity'. The role 'ecsInstanceRole' is highlighted with a red box. Below the table, the 'Roles Anywhere' section provides information on authenticating non-AWS workloads, mentioning 'Access AWS from your non AWS workloads', 'X.509 Standard', and 'Temporary credentials'.

Role name	Trusted entities	Last activity
EC2-ECR-Role	AWS Service: ec2	2 days ago
ecsInstanceRole	AWS Service: ec2	-

STEP 8: Create Security Group for ECS EC2 Instances

Go to:

EC2

→ Security Groups

→ Create security group

Enter:

Security group name: ecs-instance-sg

VPC: hiring-ecs-vpc

Inbound Rule 1 — SSH

Type: SSH

Port: 22

Source: My IP

Application Traffic

Because we are using dynamic port mapping, the ALB needs to reach dynamically assigned host ports.

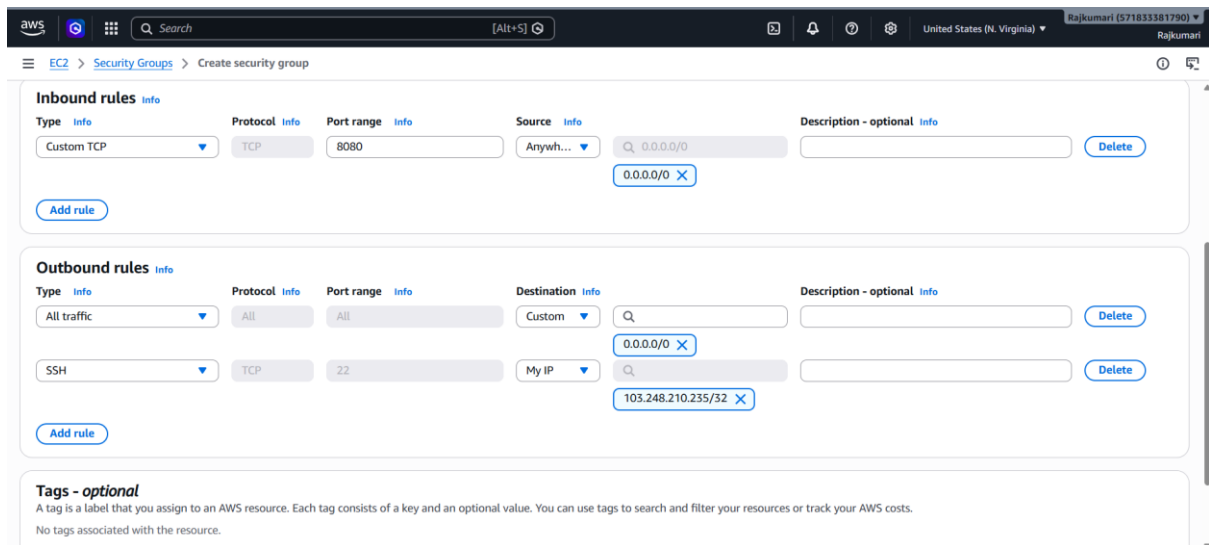
Add:

Type: Custom TCP

Port range: 32768-65535

Source: ALB Security Group

We will create the ALB security group shortly. The guide specifically recommends allowing the ALB SG to access the ECS dynamic port range rather than opening it to the internet.



STEP 9: Create ALB Security Group

Go to:

EC2

→ Security Groups

→ Create security group

Enter:

Name: hiring-alb-sg

VPC: hiring-ecs-vpc

Add inbound rule:

Type: HTTP

Port: 80

Source: 0.0.0.0/0

Optional HTTPS:

Type: HTTPS

Port: 443

Source: 0.0.0.0/0

Internet

|

| Port 80

v

ALB Security Group

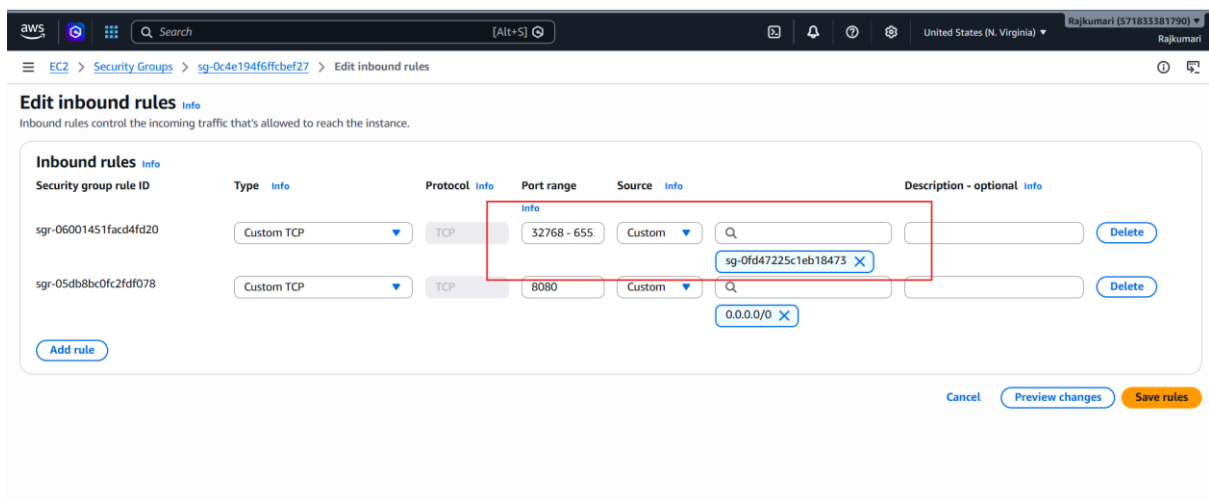
|

| Dynamic Ports

| 32768-65535

v

ECS Instance Security Group



STEP 11: Create ECS Cluster

Go to:

AWS Console

→ ECS

→ Clusters

→ Create cluster

Cluster name:

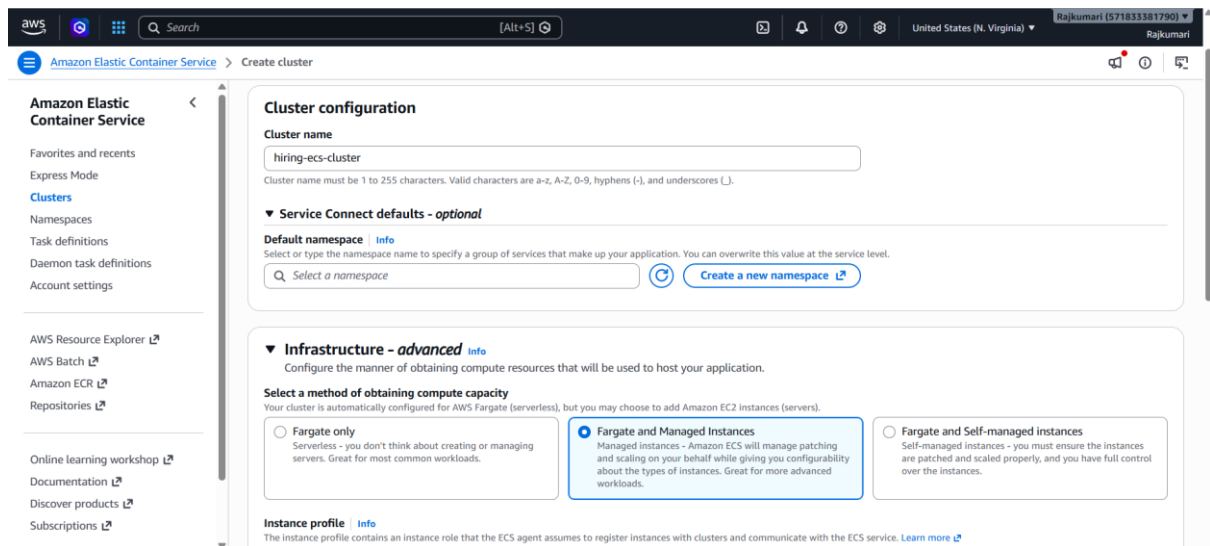
hiring-ecs-cluster

Choose EC2-based infrastructure according to the current ECS Console option.

The guide uses:

Amazon EC2 instances

Create the cluster.



STEP 12: Create EC2 Launch Template

Go to:

EC2

→ Launch Templates

→ Create launch template

Name:

hiring-ecs-launch-template

AMI

Choose:

Amazon ECS-Optimized Amazon Linux 2023

Instance Type

Start with:

t3.micro

However, 6 tasks may not fit if the application requires significant memory. If tasks fail due to insufficient CPU or memory, use a larger instance such as:

t3.small

Security Group

Select:

ecs-instance-sg

IAM Instance Profile

Select:

ecsInstanceRole

User Data

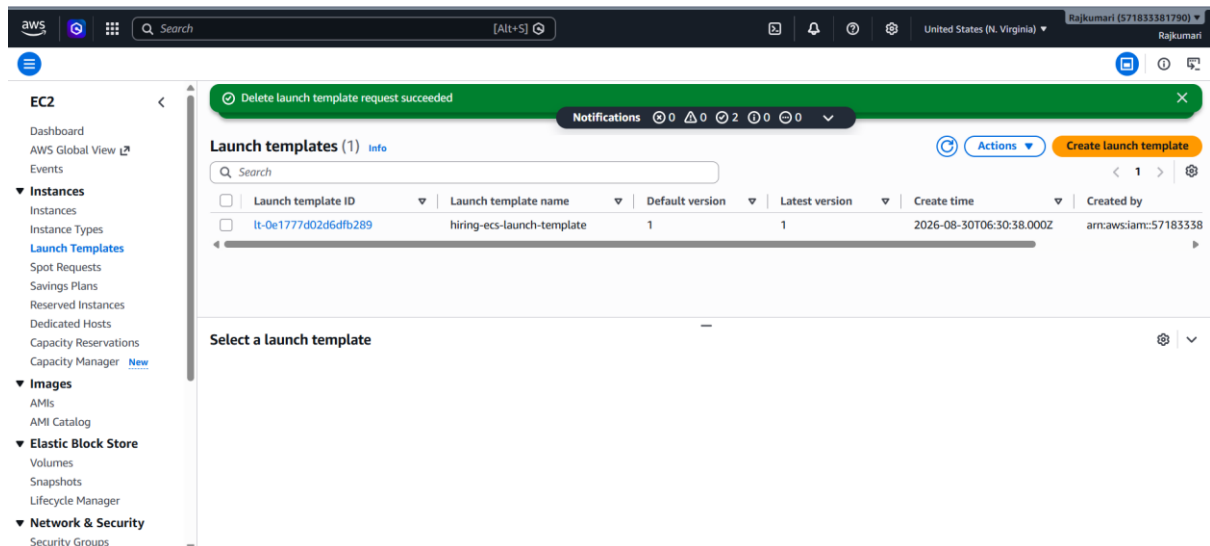
Enter:

```
#!/bin/bash
```

```
echo ECS_CLUSTER=hiring-ecs-cluster >> /etc/ecs/ecs.config
```

This makes every EC2 instance launched by the ASG join your ECS cluster.

Create the launch template.



STEP 13: Create Auto Scaling Group

Go to:

EC2

→ Auto Scaling Groups

→ Create Auto Scaling Group

Name:

hiring-ecs-asg

Select:

Launch Template:

hiring-ecs-launch-template

Click Next.

Select your VPC:

hiring-ecs-vpc

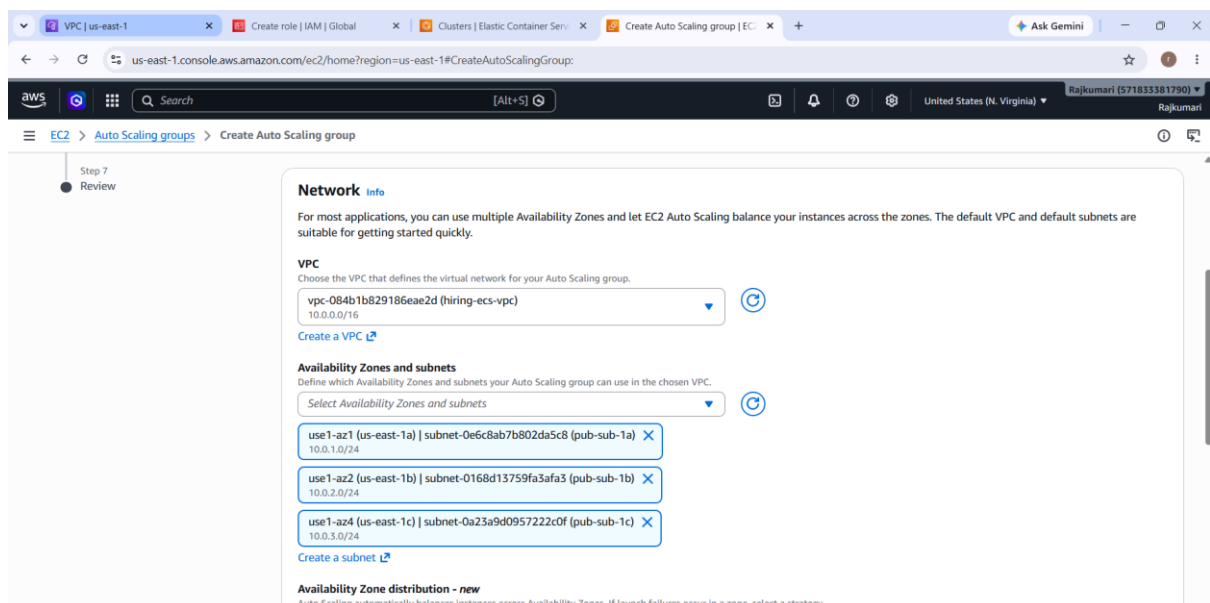
Select all three subnets:

public-subnet-1a

public-subnet-1b

public-subnet-1c

This is important for multi-AZ high availability.



STEP 14: Configure ASG Capacity

Set:

Minimum capacity: 3

Desired capacity: 3

Maximum capacity: 6

This should launch approximately one EC2 instance in each AZ.

Your architecture becomes:

AZ-1 → EC2-1

AZ-2 → EC2-2

AZ-3 → EC2-3

After creating the ASG, wait a few minutes.

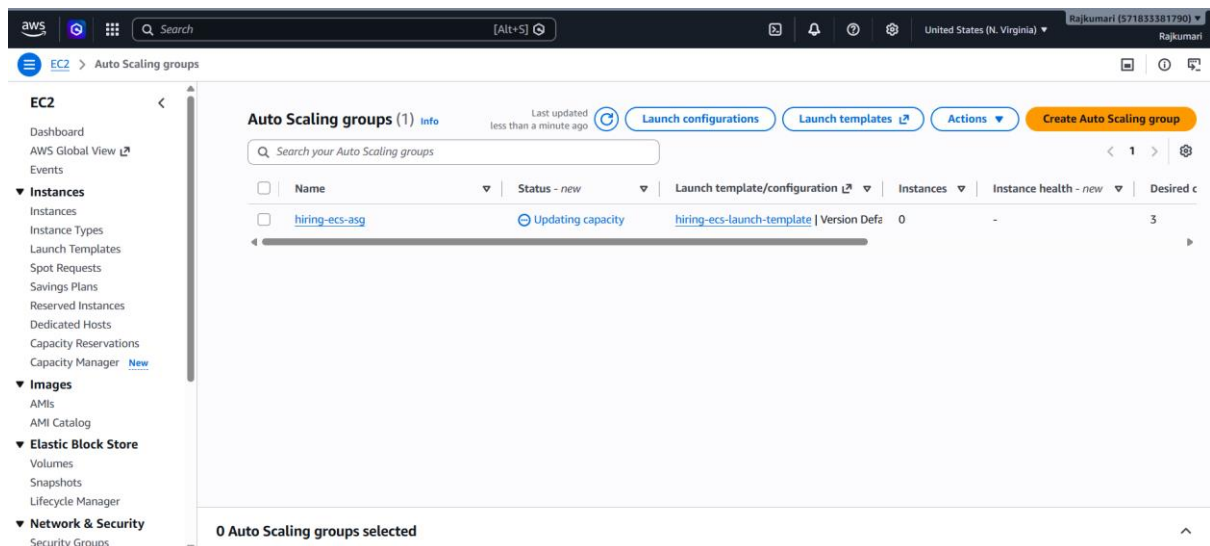
Verify:

EC2

→ Instances

You should see:

3 Running EC2 Instances



STEP 15: Verify EC2 Instances Joined ECS Cluster

Go to:

ECS

→ Clusters

→ hiring-ecs-cluster

→ Infrastructure

You should see:

3 Container Instances

If you see:

0 Container Instances

Do not continue. First fix the ECS registration issue.

STEP 16: Create Application Load Balancer

Go to:

EC2

→ Load Balancers

→ Create Load Balancer

Select:

Application Load Balancer

Configure:

Name: hiring-alb

Scheme: Internet-facing

IP address type: IPv4

VPC: hiring-ecs-vpc

Select all 3 public subnets:

public-subnet-1a

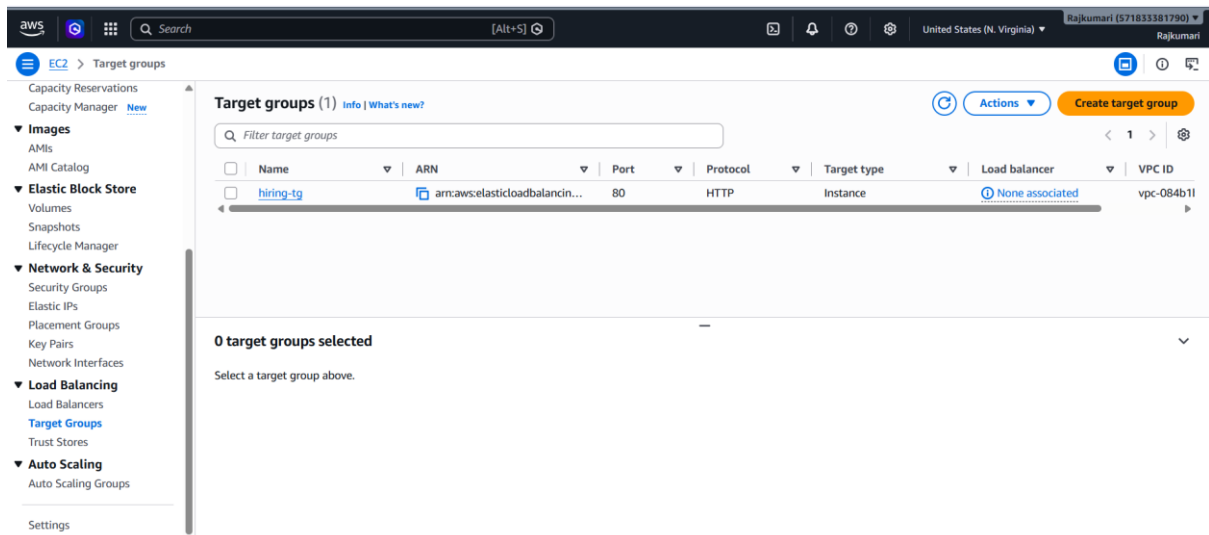
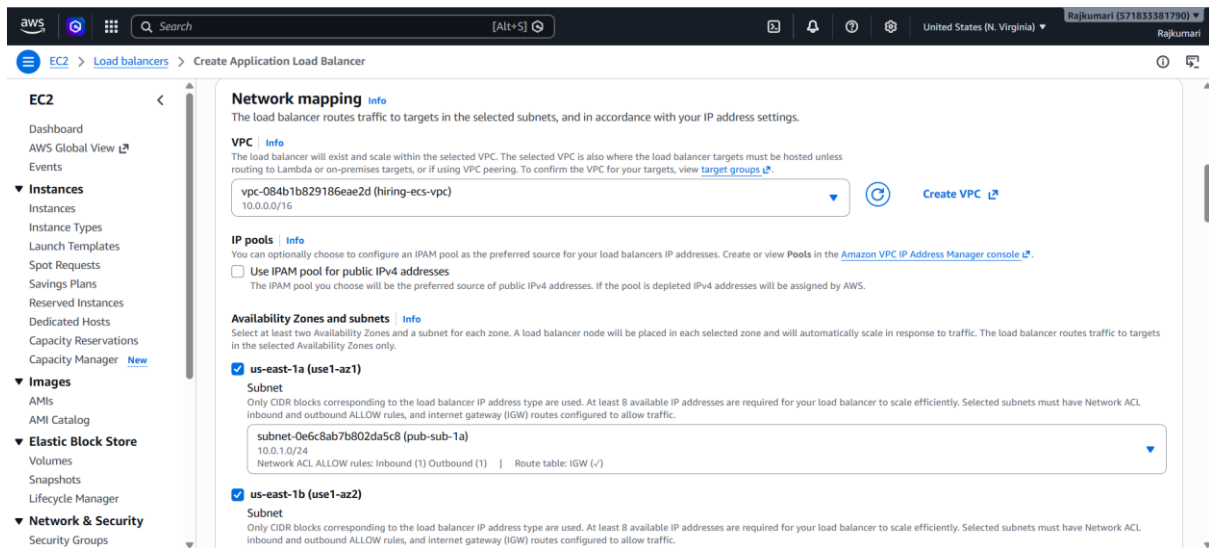
public-subnet-1b

public-subnet-1c

Select:

Security Group: hiring-alb-sg

An ALB must span multiple AZs; the guide recommends selecting all three for this design.



STEP 18: Configure ALB Listener

Create:

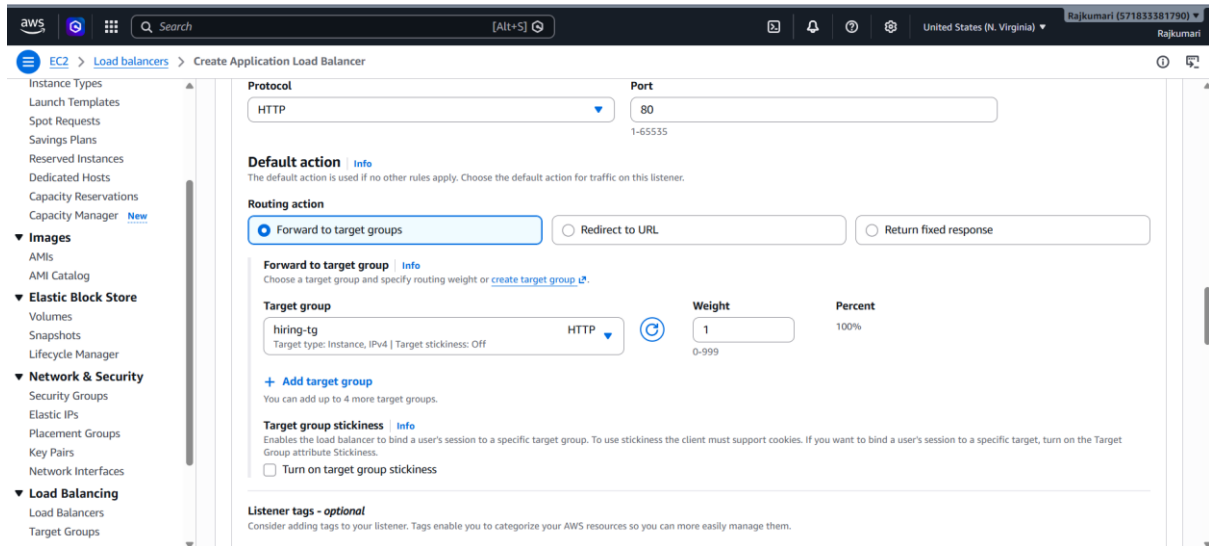
Protocol: HTTP

Port: 80

Default action:

Forward to:

hiring-tg



STEP 19: Check the Application Container Port

Before creating the task definition, verify which port the image exposes.

On your Linux server with Docker:

```
docker pull sabair0509/hiring-app:works
```

Then run:

```
docker inspect sabair0509/hiring-app:works \
--format='{{json .Config.ExposedPorts}}'
```

Possible output:

```
{"8080/tcp":{}}
```

If you get that, use:

Container Port = 8080

If you get:

```
{"80/tcp":{}}
```

Use:

Container Port = 80

Do not assume the port. Check it first.

STEP 20: Create ECS Task Definition

Go to:

ECS

→ Task Definitions

→ Create new task definition

Select:

Launch type / Infrastructure: EC2

Task family name:

hiring-task

For example:

CPU: 256

Memory: 512 MB

Remember: 6 tasks require enough EC2 CPU and memory capacity.

STEP 21: Add Container

Container name:

hiring-app

Image:

sabair0509/hiring-app:works

The screenshot shows the AWS Management Console for the Amazon Elastic Container Service. The main content area displays the details for a task definition named 'hiring-task:1'. A green banner at the top indicates that the task definition was successfully created. The task definition is currently in an 'ACTIVE' state. The console shows the task definition's ARN, the role it uses for execution, and the operating system/architecture. The 'Task size' section specifies 1,024 vCPU units and 3,072 MiB of memory. The left sidebar shows the navigation menu with 'Task definitions' selected.

ARN	Status	Time created	App environment
arn:aws:ecs:us-east-1:571833381790:task-definition/hiring-task:1	ACTIVE	August 30, 2026, 12:47 (UTC+5:30)	EC2

Task role	Task execution role	Operating system/Architecture	Network mode
-	ecsTaskExecutionRole	Linux/X86_64	awsvpc

Task size	Task memory
Task CPU: 1,024 units (1 vCPU)	3,072 MiB (3 GiB)

STEP 22: Configure Dynamic Port Mapping

This is the **most important step**.

Suppose your Docker inspection showed:

8080

Configure:

Container port: 8080

Protocol: TCP

Host port: 0

The important setting is:

Host port = 0

This enables dynamic port mapping.

For example:

Task 1 → EC2-1:32771 → Container:8080

Task 2 → EC2-1:32772 → Container:8080

Task 3 → EC2-2:32773 → Container:8080

Task 4 → EC2-2:32774 → Container:8080

Task 5 → EC2-3:32775 → Container:8080

Task 6 → EC2-3:32776 → Container:8080

If your actual application port is 80, replace 8080 with 80.

STEP 23: Create ECS Capacity Provider

Go to:

ECS

→ Clusters

→ hiring-ecs-cluster

→ Infrastructure

→ Capacity Providers

→ Create

Name:

hiring-capacity-provider

Select:

Auto Scaling Group:

hiring-ecs-asg

Enable:

Managed Scaling: ON

Target Capacity: 80

Also enable managed draining if available.

Then associate this capacity provider with:

hiring-ecs-cluster

Set it as the default capacity provider

The screenshot shows the AWS Management Console interface for creating a new Capacity Provider. The breadcrumb navigation at the top indicates the path: Amazon Elastic Container Service > Clusters > hiring-ecs-dev-cluster > Infrastructure > Capacity providers > Create. The left sidebar lists various AWS services, with 'Amazon Elastic Container Service' selected. The main content area is titled 'Basic details' and contains the following sections:

- Cluster name:** A text input field containing 'hiring-ecs-dev-cluster'.
- Capacity provider name:** A text input field containing 'hiring-capacity-provider'. Below the field, a note states: 'Up to 255 characters are allowed, including letters (upper and lowercase), numbers, underscores, and hyphens. The name cannot be prefixed with "aws", "ecs", or "fargate".'
- Scaling type:** A section titled 'Select the capacity provider scaling type' with two options:
 - ☐ **Managed Instances**: Use managed instances to have Amazon ECS manage scaling and patching of instances on your behalf.
 - ☒ **EC2 Auto Scaling**: Use an Auto Scaling group to manually configure EC2 capacity.A note below states: 'Scaling type cannot be updated after creation of capacity provider'.
- Auto Scaling group:** A section titled 'Use an existing Auto Scaling group' with a note: 'You must create an Auto Scaling group before creating a capacity provider. Use the [Amazon EC2 console](#) to create an Auto Scaling group.' Below this is a dropdown menu showing 'hiring-ecs-asg' and a refresh icon.
- Scaling policies - optional:** A section that is currently collapsed.

STEP 24: Create ECS Service

Go to:

ECS

→ Clusters

→ hiring-ecs-cluster

→ Services

→ Create

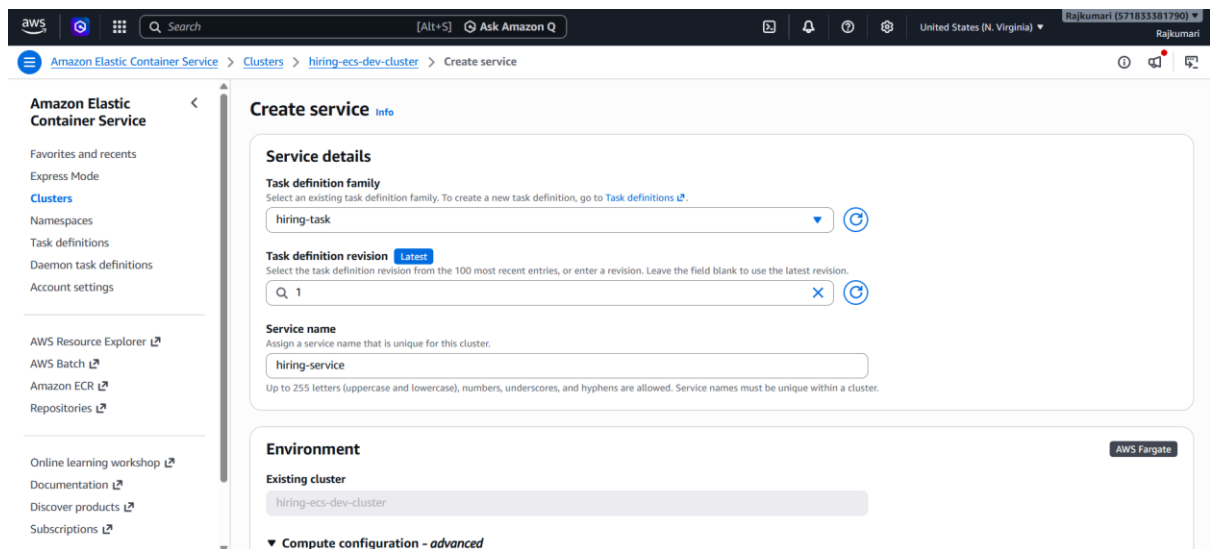
Configure:

Service name: hiring-service

Task definition: hiring-task

Compute option: Capacity Provider

Capacity provider: hiring-capacity-provider



STEP 25: Set Desired Tasks to 6

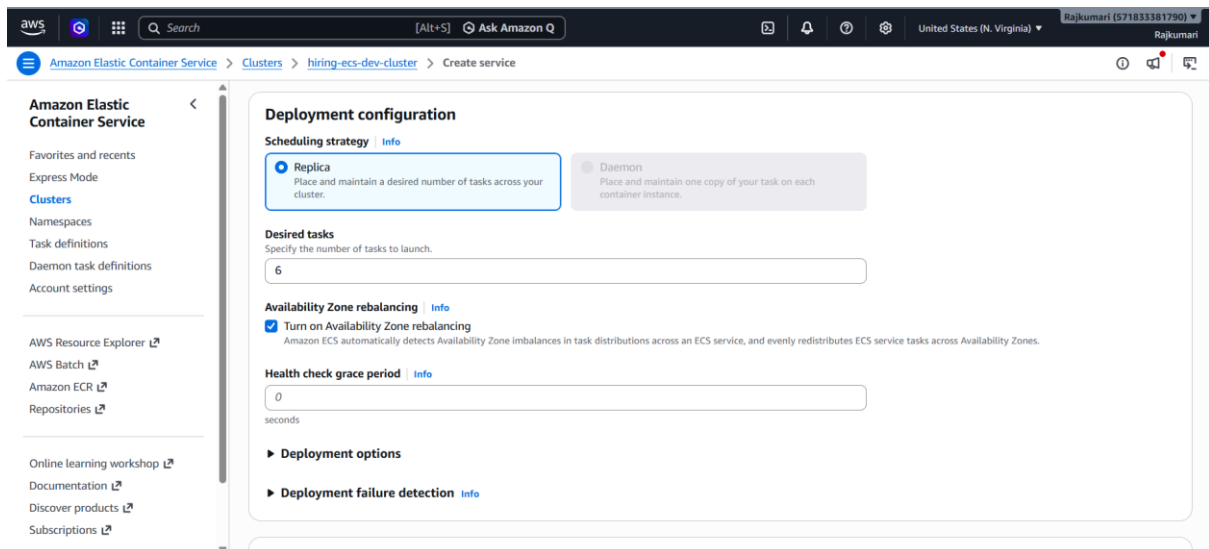
Set:

Desired tasks: 6

This is mandatory for your assignment.

Desired = 6

Running = 6



STEP 26: Configure Deployment Settings

Use:

Minimum healthy percentage: 100%

Maximum percentage: 200%

This helps ECS maintain healthy capacity during deployments.

STEP 27: Configure Task Placement

Configure task placement so tasks spread across:

1. Availability Zone

2. Instance ID

Desired distribution:

AZ-1

├── Task 1

└── Task 2

AZ-2

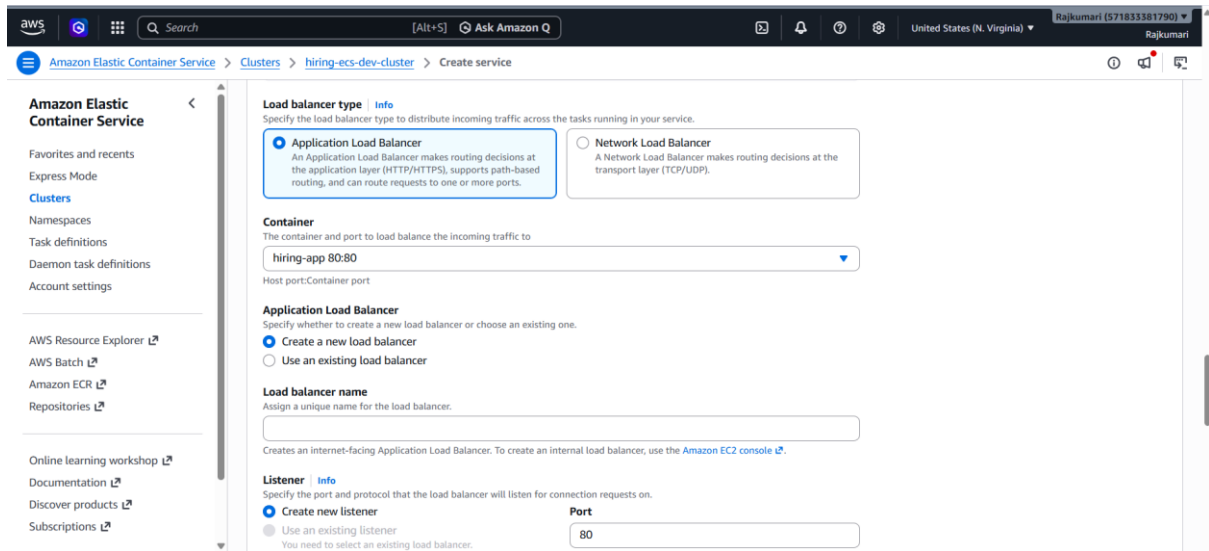
├── Task 3

└── Task 4

AZ-3

└─ Task 5

└─ Task 6



STEP 28: Configure Load Balancer in ECS Service

Select:

Load balancer: Application Load Balancer

ALB: hiring-alb

Target Group: hiring-tg

Container: hiring-app

Container Port: YOUR_ACTUAL_PORT

Example:

Container Port: 8080

Use 80 instead if that is what your Docker image exposes.

Set health check grace period:

60 seconds

Then create the service.

STEP 29: Wait for 6 Tasks

Go to:

ECS

→ Clusters

→ hiring-ecs-cluster

→ Services

→ hiring-service

You should see:

Desired: 6

Running: 6

Pending: 0

The screenshot shows the AWS Management Console interface for the 'Create service' page in Amazon Elastic Container Service (ECS). The breadcrumb navigation at the top indicates the path: Amazon Elastic Container Service > Clusters > hiring-ecs-dev-cluster > Create service. The left sidebar contains navigation links for 'Favorites and recents', 'Express Mode', 'Clusters' (highlighted), 'Namespaces', 'Task definitions', 'Daemon task definitions', 'Account settings', 'AWS Resource Explorer', 'AWS Batch', 'Amazon ECR', 'Repositories', 'Online learning workshop', 'Documentation', 'Discover products', and 'Subscriptions'. The main content area is titled 'Target group' and includes an 'Info' link. It instructs the user to 'Specify whether to create a new target group or choose an existing one that the load balancer will use to route requests to the tasks in your service.' The 'Create new target group' radio button is selected. Below this, the configuration fields are: 'Target group name' (text input), 'Protocol' (dropdown menu set to 'HTTP'), 'Port' (text input set to '8080'), 'Deregistration delay' (text input set to '300' with a note 'The amount of time to wait before the state of a deregistering target changes from draining to unused.' and 'seconds' label), 'Health check protocol' (dropdown menu set to 'HTTP'), and 'Health check path' (text input set to '/').

The screenshot shows the AWS Management Console for Amazon Elastic Container Service (ECS). The breadcrumb navigation indicates the path: Amazon Elastic Container Service > hiring-ecs-dev-cluster > Services. A green success banner at the top states: "hiring-app-service has been deployed successfully." with a "View service" button. The main heading is "hiring-ecs-dev-cluster", with a sub-header "Last updated August 30, 2026, 15:05 (UTC+5:30)". Below this is an orange button "Create with Express Mode". The "Cluster overview" section contains the following details:

Cluster ARN	Status
arn:aws:ecs:us-east-1...	Active

CloudWatch monitoring	Action logs
Default	Turned off

Services	Tasks
0 active 0 draining	0 running 0 pending

Registered container instances: -

The left sidebar lists navigation options: Favorites and recents, Express Mode, Clusters (selected), Namespaces, Task definitions, Daemon task definitions, Account settings, AWS Resource Explorer, AWS Batch, Amazon ECR, Repositories, Online learning workshop, Documentation, Discover products, and Subscriptions. The bottom navigation bar shows tabs for Services (selected), Tasks, Daemons, and Infrastruc.

STEP 29: Wait for 6 Tasks

Go to:

ECS

→ Clusters

→ hiring-ecs-cluster

→ Services

→ hiring-service

You should see:

Desired: 6

Running: 6

Pending: 0

The screenshot shows the AWS Management Console for the Amazon Elastic Container Service. The breadcrumb navigation is: Amazon Elastic Container Service > Services > hiring-app-service > Tasks > 52b6f66a77ad4f929fa9798f379f3944 > Configuration. The left sidebar shows the navigation menu with 'Amazon Elastic Container Service' selected. The main content area is titled 'Task overview' and 'Task lifecycle'. The 'Task overview' section includes: Task ID (52b6f66a77ad4f929fa9798f379f3944), Task ARN (arn:aws:ecs:us-east-1:571833381290:task-definition/hiring-task/1), Task definition: revision (hiring-task:1), CPU | memory (1 vCPU | 3 GiB), Launch type (Fargate), Last status (Running), Desired status (Running), Service (hiring-app-service), Service revision (9339179801492148054), and Started by (ecs-svc/9339179801492148054). The 'Task lifecycle' section shows a timeline of events: Created (August 30, 2026, 15:05 (UTC+5:30)), Provisioning, Network interface attached (August 30, 2026, 15:05 (UTC+5:30)), Pending, Pull started (August 30, 2026, 15:05 (UTC+5:30)), Pull completed (August 30, 2026, 15:05 (UTC+5:30)) (11 seconds), and Running (August 30, 2026, 15:05 (UTC+5:30)) (Duration: 21 minutes, 51 seconds). Below the task overview is the 'Container details for hiring-app' section, with tabs for Details, Log configuration, Restart policy, Network bindings, Docker labels and hosts, and Environment variables and nites. The 'Network bindings' tab is selected, showing a table with columns: Host port, Container port, Protocol, and External link. The table contains one entry: Host port 80, Container port 80, Protocol tcp, and External link 44.202.49.252:80 | open address.

The screenshot shows the AWS Management Console for the Amazon Elastic Container Service, specifically the 'hiring-ecs-dev-cluster'. The breadcrumb navigation is: Amazon Elastic Container Service > Clusters > hiring-ecs-dev-cluster > Tasks. The left sidebar shows the navigation menu with 'Amazon Elastic Container Service' selected. The main content area is titled 'hiring-ecs-dev-cluster' and 'Cluster overview'. The 'Cluster overview' section includes: Cluster ARN (arn:aws:ecs:us-east-1:571833381290:cluster/hiring-ecs-dev-cluster), Status (Active), CloudWatch monitoring (Default), Action logs (Turned off), Services (1 active | 0 draining), Tasks (1 running | 0 pending), and Registered container instances (-). Below the cluster overview is the 'Tasks' section, with tabs for Services, Tasks, Daemons, Infrastructure, Metrics, Scheduled tasks, Configuration - updated, Event history, and Tags. The 'Tasks' tab is selected, showing a table with columns: Task, Last status, Desired status, Task definition, Health status, and Started by. The table contains one entry: Task 52b6f66a77ad4f929fa9798f379f3944, Last status Running, Desired status Running, Task definition hiring-task:1, Health status Unknown, and Started by ecs-svc/9339179801492148054. The table has a filter bar at the top with a search box 'Filter tasks by property or value', a 'Filter desired status' dropdown set to 'Any desired status', and a 'Filter launch type' dropdown set to 'Any launch type'. There are also buttons for 'Manage tags', 'Stop', and 'Run new task'.

STEP 30: Verify Dynamic Port Mapping

Go to:

ECS

→ Cluster

→ Service

→ Tasks

→ Select a Task

→ Containers

You should see something like:

Container Port	Host Port
8080	32771
8080	32772
8080	32773

The key concept is:

Container Port = Same

Host Port = Different

That proves dynamic port mapping.

30. Configure task placement

This is another important part.

Under task placement, use a strategy that spreads tasks across AZs and instances.

Conceptually:

Spread by:

attribute:ecs.availability-zone

Then spread by:

instanceId

This gives you something like:

AZ-1

├── Task 1

└── Task 2

AZ-2

|— Task 3

|— Task 4

AZ-3

|— Task 5

|— Task 6

This is much better than allowing all six tasks to accidentally land on one EC2 instance.

31. Configure health check grace period

If your application needs some startup time, set:

Health check grace period:

60 seconds

For slower applications:

120 seconds

This prevents ECS from immediately killing a task while the application is still starting.

32. Configure load balancer

Select:

Application Load Balancer:

hiring-alb

Listener:

HTTP : 80

Target group:

hiring-tg

Container:

hiring-app

Port:

8080

Again, use the actual container port.

33. Update ECS instance security group

Your security groups should look conceptually like:

Internet

|

| TCP 80

v

ALB Security Group

|

| Dynamic ports

| 32768-65535

v

ECS Instance Security Group

|

v

Docker containers

The ECS instance SG should allow the ALB SG to access the dynamic port range.

For example:

Inbound

Type: Custom TCP

Port: 32768-65535

Source: hiring-alb-sg

34. Create the service

Click:

Create

ECS should begin launching:

6 tasks

Watch:

ECS → Cluster → hiring-ecs-cluster → Services → hiring-service

You should eventually see:

Desired: 6

Running: 6

Pending: 0

The ideal result is:

Desired count = 6

Running count = 6

35. Check the six tasks

Go to:

ECS → Cluster → Service → Tasks

You should see:

Task 1 RUNNING

Task 2 RUNNING

Task 3 RUNNING

Task 4 RUNNING

Task 5 RUNNING

Task 6 RUNNING

36. Verify AZ distribution

Check each task's EC2 container instance.

You want something like:

AZ-1

└─ EC2-1

| └─ Task 1

| └─ Task 2

AZ-2

└─ EC2-2

| └─ Task 3

| └─ Task 4

AZ-3

└─ EC2-3

└─ Task 5

└─ Task 6

This demonstrates the **high availability requirement**.

37. Verify dynamic port mapping

Go to:

ECS → Cluster → Service → Tasks → Task → Containers

You should see something similar to:

Container port	Host port
----------------	-----------

8080	32771
------	-------

8080	32772
------	-------

For another task:

Container port	Host port
----------------	-----------

8080	32773
------	-------

Notice:

Container port = same

Host port = different

That's dynamic port mapping.

38. Verify Target Group

Go to:

EC2 → Target Groups → hiring-tg → Targets

You should see approximately:

Instance	Port	Health
i-1111111	32771	Healthy
i-1111111	32772	Healthy
i-2222222	32773	Healthy
i-2222222	32774	Healthy
i-3333333	32775	Healthy
i-3333333	32776	Healthy

This is one of the **best screenshots to take for your assignment**.

It proves:

- 6 tasks
 - multiple instances
 - dynamic ports
 - ALB registration
 - healthy targets
-

39. Test the application

Go to:

EC2 → Load Balancers → hiring-alb

Copy:

DNS name

It will look similar to:

hiring-alb-123456789.us-east-1.elb.amazonaws.com

Open:

<http://hiring-alb-123456789.us-east-1.elb.amazonaws.com>

You should get your hiring application.

40. Test from Linux

You can also run:

```
curl http://YOUR-ALB-DNS-NAME
```

For example:

```
curl http://hiring-alb-123456789.us-east-1.elb.amazonaws.com
```

41. Test load balancing

Run:

```
for i in {1..20}
```


do

```
curl -s http://YOUR-ALB-DNS-NAME
```

echo

done

The ALB will distribute requests across healthy targets.

42. Test high availability

This is a very good demonstration for your assignment.

Suppose you have:

AZ-1 → Task 1, Task 2

AZ-2 → Task 3, Task 4

AZ-3 → Task 5, Task 6

Stop one task.

For example:

Task 1 → STOPPED

ECS should detect that the service has:

Desired = 6

Running = 5

and launch a replacement.

Eventually:

Desired = 6

Running = 6

This demonstrates ECS service self-healing.

43. Test EC2 failure

For an even stronger demonstration, terminate one ECS EC2 instance.

For example:

AZ-1

EC2-1

After termination, ECS/Auto Scaling should replace the EC2 capacity, and ECS should replace tasks that were lost.

AWS recommends using Auto Scaling across multiple AZs for availability, and ECS capacity providers can manage EC2 capacity scaling.

44. Important: "At least 6 tasks at any point"

There is a subtle difference between:

Desired count = 6

and a strict guarantee of:

6 healthy tasks at every instant

During an unexpected failure, there can temporarily be fewer than six running tasks while ECS replaces them.

For a stronger HA design, you need **spare capacity**.

For example:

Desired tasks = 6

EC2:

AZ-1 → 2 tasks

AZ-2 → 2 tasks

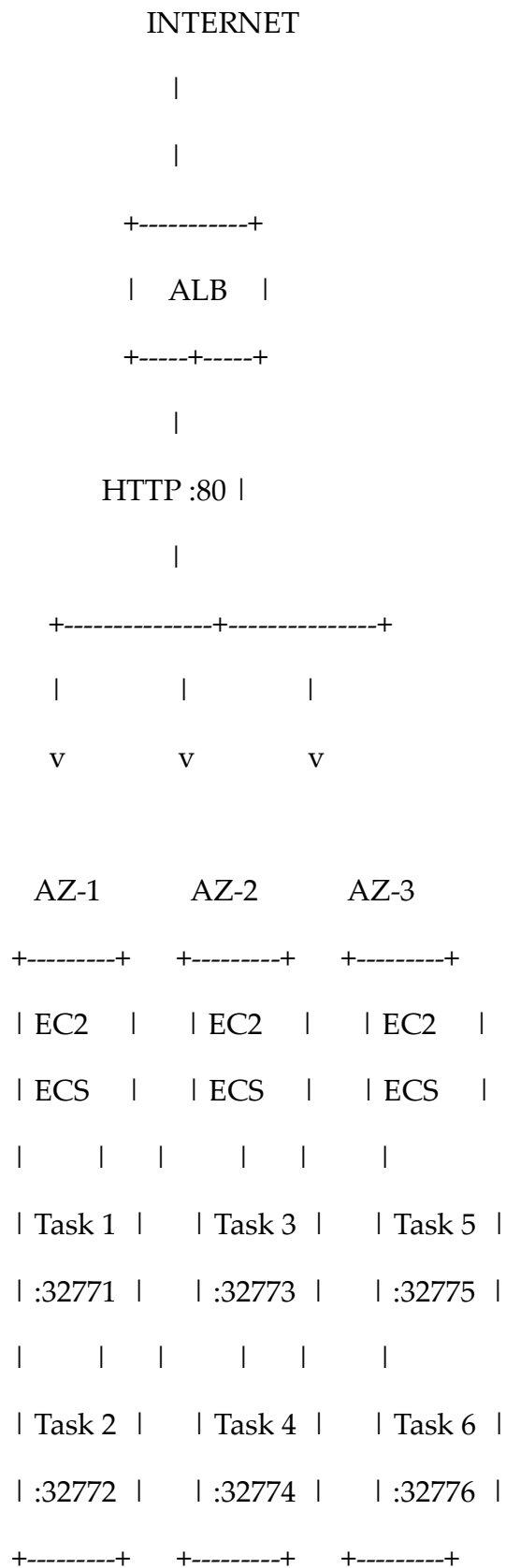
AZ-3 → 2 tasks

and enough unused EC2 capacity to immediately replace failed tasks.

AWS specifically recommends maintaining capacity headroom because scaling and recovery take time.

45. Recommended final architecture

For your assignment, I would use:



|

v

Docker Image

sabair0509/hiring-app:works

46. Final verification checklist

Before you submit the assignment, verify all of these:

- [✓] ECS cluster created
- [✓] EC2 launch type used
- [✓] ECS capacity provider created
- [✓] Auto Scaling Group created
- [✓] ASG spans multiple AZs
- [✓] At least 3 ECS EC2 instances
- [✓] ALB created
- [✓] ALB spans multiple AZs
- [✓] Target group created
- [✓] ECS task definition created
- [✓] Image = sabair0509/hiring-app:works
- [✓] Container port verified
- [✓] Host port = 0
- [✓] Dynamic port mapping enabled
- [✓] ECS service created
- [✓] Desired count = 6
- [✓] Running count = 6

[✓] Tasks distributed across AZs

[✓] Target group shows 6 healthy targets

[✓] ALB DNS opens application

[✓] ECS replaces stopped task

[✓] EC2 Auto Scaling provides replacement capacity

Amazon Elastic Container Service

Protection status: Turned off

Protection duration: -

Containers (1)

Container name	Container runtime ID	Image URI	Image Digest	Status	Health status	CPU
hiring-app	52b6f66a77ad4f9...	sabair050...	sha256:58...	Running	Unknown	0

Container details for hiring-app

Details

Image URI: sabair0509/hiring-app:works

Essential: Yes

Command: -

EC2 > Instances

Instances (6)

Instance state: running

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS
	i-Qa8377736867433b5	Running	t2.small	2/2 checks passed	us-east-1b	-
	i-0023dc51bf27fa2ad	Running	t2.small	2/2 checks passed	us-east-1b	-
	i-0c97e6254549c8416	Running	t2.small	2/2 checks passed	us-east-1a	-
	i-00ce59020cfdcb5b	Running	t2.small	2/2 checks passed	us-east-1c	-
	i-09013c0c5ee54216e	Running	t2.small	2/2 checks passed	us-east-1c	-
	i-0f647a5638ceebd61	Running	t2.small	2/2 checks passed	us-east-1a	-

All instances

Instance alarm states: In alarm: None

Service health: EC2 service status: Operating normally

Health events affecting EC2 instances: Open events: None, Upcoming events: None

```

root@ip-10-0-0-147:~
brajk@RajKumari MINGW64 ~/Downloads
$ ssh -i server-2.pem ec2-user@3.239.17.220
ssh: connect to host 3.239.17.220 port 22: Connection timed out

brajk@RajKumari MINGW64 ~/Downloads
$ ssh -i server-2.pem ec2-user@3.239.17.220
The authenticity of host '3.239.17.220 (3.239.17.220)' can't be established.
ED25519 key fingerprint is SHA256:o8xGwo1rRfH5Z2wqLoyxFHEclVlPVBwrwQTBKxia6Hg.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.239.17.220' (ED25519) to the list of known hosts.

  _ | _ | _ |
  _ | ( _ \ _ \
 _ _ | \ _ | _ _ |

Amazon ECS-Optimized Amazon Linux AMI

For documentation, visit http://aws.amazon.com/documentation/ecs
1 package(s) needed for security, out of 1 available
Run "sudo yum update" to apply all updates.
[ec2-user@ip-10-0-0-147 ~]$ sudo su -
[root@ip-10-0-0-147 ~]# |

```

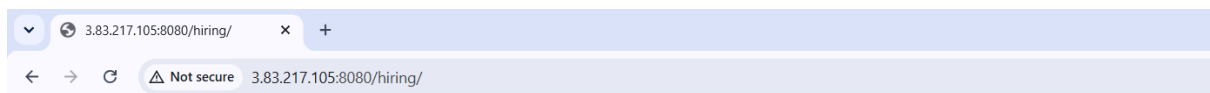
```
ec2-user@ip-10-0-2-167:~$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
sabair0509/hiring-app	works	b49fe6b94ba1	2 years ago	426MB
amazon/amazon-ecs-agent	latest	f5668fdd3a82	5 years ago	61.5MB
amazon/amazon-ecs-pause	0.1.0	2ff4c694d4df	5 years ago	954kB

```
ec2-user@ip-10-0-2-167 ~]$ |
```

```
root@ip-10-0-0-230:~  
[root@ip-10-0-0-230 ~]# docker ps  
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES  
[root@ip-10-0-0-230 ~]# docker images  
REPOSITORY          TAG         IMAGE ID            CREATED        SIZE  
amazon/amazon-ecs-agent   latest     f5668fdd3a82       5 years ago   61.5MB  
amazon/amazon-ecs-pause   0.1.0      2ff4c694d4df       5 years ago   954kB  
[root@ip-10-0-0-230 ~]# |
```

```
ec2-user@ip-10-0-2-167:~  
[ec2-user@ip-10-0-2-167 ~]$ docker images  
REPOSITORY          TAG         IMAGE ID            CREATED        SIZE  
sabair0509/hiring-app   works      b49fe6b94ba1       2 years ago   426MB  
amazon/amazon-ecs-agent   latest     f5668fdd3a82       5 years ago   61.5MB  
amazon/amazon-ecs-pause   0.1.0      2ff4c694d4df       5 years ago   954kB  
[ec2-user@ip-10-0-2-167 ~]$ docker ps -a  
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES  
ea271aa44367   sabair0509/hiring-app:works   "catalina.sh run"   9 minutes ago   Up 9 minutes   0.0.0.0:8080->8080/tcp, :::8080->8080/tcp   hiring-app  
e4b37424b446   amazon/amazon-ecs-agent:latest   "/agent"           4 hours ago    Up 4 hours (healthy)                   ecs-agent  
[ec2-user@ip-10-0-2-167 ~]$ |
```



Welcome to Techie Horizon

